

Curvas Paramétricas — Bézier e Hermite

Trabalho Prático 02 — Computação Gráfica

Autor: Lucas Gualtieri · 2026

Documentação técnica do TP 02. Estende o TP 01 (matriz de pixels com retas, circunferência, recorte e transformações 2D) com **duas curvas paramétricas vistas em sala**: a curva de **Bézier** (avaliada pelo algoritmo de **De Castelja**) e a curva de **Hermite cúbica**.

1. Organização do código

O projeto é uma aplicação web (Vite + TypeScript) que rasteriza primitivas diretamente sobre um *buffer* de pixels do `<canvas>`. O código é dividido em módulos com responsabilidades bem separadas (todos em `src/`):

Módulo	Responsabilidade
<code>types.ts</code>	Tipos compartilhados: <code>Point</code> , <code>VectorIJ</code> , <code>Color</code> , <code>Rect</code> , <code>Tool</code> , <code>UIState</code> .
<code>Algorithms.ts</code>	Algoritmos puros de rasterização e geometria — recebem um <code>PixelWriter (x, y, color) => void</code> e não conhecem o canvas. É aqui que vivem as curvas.
<code>Homogeneous2D.ts</code>	Matrizes homogêneas 3×3 (rotação, escala, translação) e suas operações.
<code>CanvasManager.ts</code>	Dono do <i>buffer</i> de pixels: <code>setPixel</code> , limpeza, grade e a conversão tela → coordenadas lógicas (<i>picking</i>).
<code>InputHandler.ts</code>	Traduz eventos de mouse/toque em coordenadas de pixel do canvas.
<code>UIManager.ts</code>	Constrói a barra lateral, emite o <code>UIState</code> e dispara callbacks (<code>onFinishCurve</code> , <code>onClear</code>).
<code>main.ts</code>	Orquestrador . Mantém a lista de primitivas, o fluxo de cliques, o <i>pipeline</i> de transformação e o laço de redesenho.

1.1. Fluxo entrada → transformação → rasterização

```
clique (InputHandler)
  → coordenada de pixel do canvas
  → CanvasManager.canvasPixelToLogical (inverte R·S e tira a translação)
  → main.ts: acumula pontos de controle (curveControlPoints)
  → finishCurve(): vira uma Primitive { type:"curve", kind, controlPoints }
```

```
a cada quadro, redrawScene():
  → transformPoints(): p_screen = R·S·(p + t)      (Homogeneous2D)
  → Algorithms.drawBezier / drawHermite            (amostra e rasteriza)
  → CanvasManager.setPixel                          (escreve no buffer)
```

As curvas reutilizam a infraestrutura do TP 01: a curva é **amostrada** em `segments + 1` pontos e os pontos consecutivos são ligados por `drawLineBresenham` (a mesma rasterização de retas do trabalho anterior).

1.2. Onde cada curva foi implementada

- `src/Algorithms.ts` — núcleo matemático (puro, testável, sem UI): `lerp`, `deCasteljau`, `evalHermite`, `sampleBezier` / `sampleHermite`, `drawBezier` / `drawHermite` / `drawPolyline`.
- `src/main.ts` — coleta de pontos de controle (`curveControlPoints`), `finishCurve()` e `drawCurve()` (transforma os pontos e desenha curva + polígono/alças + marcadores).
- `src/UISManager.ts` — botões *Bézier/Hermite*, *slider segments* e botão *Finalizar curva*.
- `src/types.ts` — `Tool` ganha `"bezier" / "hermite"`; `UIState` ganha `curveSegments`.

2. Modelo matemático

Uma **curva paramétrica** é uma função $P(t)$ que mapeia um parâmetro escalar $t \in [0, 1]$ num ponto do plano. Em vez de descrever y em função de x (o que impede curvas verticais ou laços), descrevemos x e y **separadamente** em função de t . Tanto Bézier quanto Hermite são curvas **polinomiais**: cada coordenada é um polinômio em t .

2.1. Curva de Bézier — algoritmo de De Casteljau

Dados $n+1$ **pontos de controle** $P_0, P_1, \dots, P_n$, a curva de Bézier de grau n é

$$B(t) = \sum_{i=0}^n \frac{n!}{i!(n-i)!} (1-t)^{n-i} t^i P_i, \quad t \in [0, 1].$$

Os coeficientes $b_{i,n}(t)$ são os **polinômios de Bernstein**. Em vez de avaliá-los diretamente, usamos o **algoritmo de De Casteljau**, numericamente mais estável e geometricamente intuitivo: ele é uma sequência de **interpolações lineares**.

Definimos $P_i^{(0)} = P_i$ e iteramos:

$$P_i^{(k)} = (1-t)P_i^{(k-1)} + tP_{i+1}^{(k-1)}, \quad k = 1 \dots n, \quad i = 0 \dots n-k.$$

Após n níveis resta um único ponto $P_0^{(n)} = B(t)$. Esquema para $n = 3$:

P_0	P_1	P_2	P_3	(nível 0 — pontos de controle)
Q_0	Q_1	Q_2		$Q_i = \text{lerp}(P_i, P_{i+1}, t)$
R_0	R_1			$R_i = \text{lerp}(Q_i, Q_{i+1}, t)$
S_0				$S_0 = \text{lerp}(R_0, R_1, t) = B(t)$

Implementação (`deCasteljau` em `Algorithms.ts`): copiamos os pontos de controle para um vetor mutável e, a cada nível, substituímos cada posição pela interpolação dela com a vizinha,

encurtando o vetor em 1 até sobrar um ponto.

Propriedades relevantes:

- **Interpolação dos extremos:** $B(0) = P_0$ e $B(1) = P_n$ — a curva passa pelo primeiro e pelo último ponto de controle, mas apenas *aproxima* os intermediários.
- **Fecho convexo:** a curva fica contida no fecho convexo dos pontos de controle.
- **Invariância afim:** aplicar uma transformação afim M aos pontos de controle e depois avaliar dá o mesmo resultado que avaliar e depois transformar. **Isso é o que nos permite transformar apenas os poucos pontos de controle** e só então amostrar a curva — exatamente o que `drawCurve` faz em `main.ts`.

2.2. Curva de Hermite cúbica

A curva de Hermite é definida por **dois pontos** P_0, P_1 e **duas tangentes** (vetores velocidade) T_0, T_1 nesses pontos. A forma cúbica é

$$P(t) = H_{00}(t)P_0 + H_{10}(t)T_0 + H_{01}(t)P_1 + H_{11}(t)T_1,$$

com os **polinômios de base de Hermite**:

$$\begin{aligned} H_{00}(t) &= 2t^3 - 3t^2 + 1 & H_{10}(t) &= t^3 - 2t^2 + t \\ H_{01}(t) &= -2t^3 + 3t^2 & H_{11}(t) &= t^3 - t^2 \end{aligned}$$

Eles são construídos para satisfazer as **condições de contorno**:

$$P(0) = P_0, \quad P(1) = P_1, \quad P'(0) = T_0, \quad P'(1) = T_1.$$

Ou seja, a curva **interpola** os dois pontos e sai/chega com as tangentes prescritas — útil quando se quer controlar a direção e a "força" da curva nas pontas.

Entrada por alças. Tangentes são difíceis de clicar diretamente, então a interface usa **alças**: o usuário clica o ponto P_0 e depois uma alça H_0 ; a tangente é a diferença $T_0 = H_0 - P_0$ (idem para P_1, H_1). Internamente a primitiva guarda os quatro pontos $[P_0, H_0, P_1, H_1]$.

Tangentes sob transformação. Pontos são posições (transformam como $Ap + b$), mas tangentes são **vetores** (transformam só pela parte linear, Av , sem a translação). Guardando a alça como **posição** e calculando a tangente como diferença **depois** de transformar, obtemos automaticamente o comportamento correto:

$$T_0' = H_0' - P_0' = (AH_0 + b) - (AP_0 + b) = A(H_0 - P_0) = AT_0.$$

Por isso `drawCurve` transforma os quatro pontos como posições e só então recalcula as tangentes.

2.3. Amostragem e rasterização

Ambas as curvas são contínuas, mas a tela é discreta. Avaliamos a curva em $t_k = k/N$ para $k = 0, 1, \dots, N$ (onde $N = \text{segments}$), obtendo $N+1$ pontos, e ligamos pontos consecutivos com **Bresenham**. Quanto maior N , mais fina a aproximação (refinamento controlado pelo *slider* `segments`).

3. Manual de uso

3.1. Executando

```
npm install      # instala dependências
npm run dev      # inicia o servidor de desenvolvimento (abra a URL exibida)
npm run build    # checagem de tipos + build de produção
```

macOS: se o build acusar "*library load disallowed by system policy*" no `lightningcss`, remova a quarentena do binário nativo: `xattr -dr com.apple.quarantine node_modules/lightningcss-darwin-arm64/`.

3.2. Desenhando uma curva de Bézier

1. Na barra lateral, em **Tool Selector**, clique em **Bézier**.
2. Clique no canvas para adicionar **pontos de controle** (quantos quiser, ≥ 2). O polígono de controle (linhas cinza) e os marcadores aparecem em tempo real.
3. Clique em **Finalizar curva** (painel *Curvas Paramétricas*) para confirmar.
4. Ajuste o *slider* **segments** para refinar/suavizar a curva.

3.3. Desenhando uma curva de Hermite

1. Selecione a ferramenta **Hermite**.
2. Dê **4 cliques** nesta ordem:
 1. **P0** — ponto inicial;
 2. **alça de P0** — define a tangente T_0 (direção/força de saída);
 3. **P1** — ponto final;
 4. **alça de P1** — define a tangente T_1 (direção/força de chegada).
3. A curva é finalizada **automaticamente** no 4º clique. As alças aparecem como linhas-guia ligando cada ponto à sua alça.

3.4. Transformações e demais controles

- **dx, dy, angle, sx, sy** (painel *Transformations*) aplicam translação, rotação e escala a **toda a cena** — as curvas acompanham (invariância afim).
- **pixel size** amplia os blocos lógicos; **Grid** mostra eixos e origem.

- **Clear Matrix** limpa tudo, inclusive uma curva em construção.
- Trocar de ferramenta **descarta** a curva ainda não finalizada.

3.5. Legenda de cores

Cor	Significado
Magenta/roxo	a curva paramétrica rasterizada
Cinza	polígono de controle (Bézier) / alças de tangente (Hermite)
Amarelo	pontos de controle de uma curva já finalizada
Branco	marcadores da curva em construção

4. Referências e autoria

Os algoritmos foram implementados pelo autor com base nas seguintes referências:

1. **P. de Castelja** (1959/1963), Citroën — algoritmo de subdivisão recursiva por interpolação linear para curvas de Bézier.
2. **P. Bézier** — curvas de Bézier e a base de Bernstein.
3. **Foley, J. D.; van Dam, A.; Feiner, S. K.; Hughes, J. F.** — *Computer Graphics: Principles and Practice*. Addison-Wesley. (Bases de Bézier/Bernstein e de Hermite; matriz de Hermite e condições de contorno.)
4. **Hearn, D.; Baker, M. P.** — *Computer Graphics with OpenGL*. (Amostragem e rasterização de curvas paramétricas.)
5. Notas de aula da disciplina de Computação Gráfica, módulo *Curvas Paramétricas*.

A rasterização de segmentos (Bresenham), as transformações homogêneas e a infraestrutura de canvas reaproveitam o código do **TP 01** deste mesmo projeto.